

Aplica la Sesión 12 a tu proyecto

Continúa tu documento de planeamiento (el de la PA3) y aplica Entity Framework Core, LINQ, procedimientos almacenados y AJAX a tu propio sistema.

Desarrollo de Aplicaciones Básico

Alumno: _____ Fecha de entrega: _____

Esta es la continuación de tu proyecto. En la PA3 elegiste tu sistema y decidiste su arquitectura (escritorio + web, o puro web). Ahora vas a APLICAR lo que vimos en la Sesión 12 a la parte web de tu proyecto, y sumar la evidencia a tu documento de planeamiento. No empiezas un documento nuevo: agregas una sección más al que ya tienes.

1. Recuerda tu decisión de la PA3

En la PA3 elegiste una de estas dos rutas para tu proyecto. Escribe cuál elegiste y por qué:

- **Ruta A • Escritorio + Web:** el CRUD y la operación maestro-detalle transaccional (por ejemplo Venta → DetalleVenta) se hacen en una aplicación de escritorio (Windows Forms) que alimenta la base de datos; la web se conecta a esa MISMA base y sirve para mostrar el catálogo y para que al menos un cliente se loguee y compre algo.
- **Ruta B • Puro Web:** todo se resuelve en la web (como el ejemplo de la Sesión 12): el CRUD, la operación maestro-detalle y la compra del cliente.

IMPORTANTE elijas la ruta que elijas, **la parte web SIEMPRE aplica la Sesión 12** (EF Core + LINQ + AJAX). Lo único que cambia entre A y B es DÓNDE haces el CRUD y la operación maestro-detalle: en escritorio o en web.

2. Qué debes aplicar y justificar

De la Sesión 12 tú DECIDES qué usar en la parte web de tu proyecto: puedes trabajar con EF Core, con LINQ, con procedimientos almacenados, o combinarlos. Lo importante es que justifiques tu elección y muestres la evidencia aplicada a TU sistema. Por cada tecnología que uses, entrega lo siguiente:

Si usas Entity Framework Core

- Muestra tu modelo (las entidades o el DbContext) o un método que lea o grabe datos con EF Core, tomado de TU proyecto, con una breve explicación.

Si usas LINQ

- Muestra una consulta LINQ real de tu proyecto (por ejemplo un listado, un filtro o un reporte) y explica qué resuelve.

Si usas procedimientos almacenados

- Comparte el SP que creaste (el script CREATE PROCEDURE) y además cómo lo invocas desde tu proyecto (el FromSql o ExecuteSqlRaw). Ambas cosas: el SP y la invocación.

AJAX (obligatorio para todos)

- Sin importar la ruta ni las tecnologías que elijas, tu proyecto debe tener al menos una funcionalidad con AJAX. Muestra el handler (OnGet...) que devuelve JSON y el fetch de la vista, y explica qué se actualiza sin recargar la página.

OJO no tienes que usar las tres tecnologías. Puedes hacer **todo con procedimientos almacenados**, todo con LINQ, o combinar. Lo que importa es que lo justifiques y lo apliques a tu proyecto. El **AJAX** sí es obligatorio en todos los casos.

3. Requisitos mínimos (revísalos antes de entregar)

- La parte web accede a los datos con lo visto en la Sesión 12 (EF Core / LINQ / procedimientos almacenados, según tu decisión).
- Al menos una funcionalidad usa AJAX (obligatorio).
- Cada tecnología que uses viene con su justificación escrita y su evidencia (código de tu proyecto).
- Se respeta la regla de oro: la vista/UI nunca consulta la base de datos directamente; siempre pasa por la capa de datos.

4. Qué entregar y en qué formato

- Agrega a tu documento de la PA3 una nueva sección titulada "Parte 4 — Aplicación de la Sesión 12".
- Dentro de esa sección: tu ruta elegida, la justificación de qué usas de la Sesión 12, y las capturas de código (SP, LINQ, EF Core y AJAX) con una breve explicación de cada una.
- Adjunta también tu proyecto actualizado (comprimido en .zip).
- Documento en Word o PDF, con tu nombre, el nombre de tu proyecto y la fecha.

RECORDAR: la UI nunca habla directo con la base de datos. Tu vista llama a un método (o a un handler AJAX), y ese método usa EF Core, LINQ o un procedimiento almacenado. La presentación y el acceso a datos quedan separados.

EJEMPLO RESUELTO (referencia)

Proyecto VentasMaster

Así se vería la Parte 4 ya resuelta para el proyecto VentasMaster (tienda). Es solo un modelo de referencia: en tu entrega reemplaza estas capturas por las de TU proyecto y explica cada una con tus palabras.

Decisión y justificación general

Para VentasMaster elegí una solución puro web (Ruta B). El acceso a datos lo resuelvo con EF Core como base, uso LINQ para las consultas del día a día, y un procedimiento almacenado para el reporte de ventas por fecha (porque agrupa y suma muchos registros, y prefiero que esa lógica viva en la base de datos). El AJAX lo aplico en el registro de ventas, para buscar un producto y mostrar su precio y stock sin recargar la página.

A. Entity Framework Core — grabar la venta (maestro-detalle)

Justificación: uso EF Core para el CRUD de Productos y para grabar la Venta con su Detalle en una sola transacción (todo o nada). Si algo falla, no se graba nada.

Método GrabarVentaAsync en la capa de datos (EF Core)

using var tx = await _ctx.Database.BeginTransactionAsync();
// se graba primero la cabecera para obtener el IdVenta
_ctx.Ventas.Add(venta);
await _ctx.SaveChangesAsync();
// luego cada línea del detalle apuntando a esa venta
foreach (var d in detalles)
{
d.IdVenta = venta.IdVenta;
_ctx.DetalleVentas.Add(d);
}
await _ctx.SaveChangesAsync();
await tx.CommitAsync(); // confirma todo o nada

B. LINQ — historial de compras del cliente

Justificación: es una consulta frecuente y sencilla; con LINQ queda clara y el compilador me avisa si me equivoco en un nombre.

Método HistorialClienteAsync (LINQ to Entities)

// consulta con LINQ: trae las ventas del cliente con su detalle
var historial = await _ctx.Ventas
.Include(v => v.Detalles)

```

.Where(v => v.IdCliente == idCliente)
.OrderByDescending(v => v.Fecha)
.ToListAsync();

```

C. Procedimiento almacenado — reporte de ventas por fecha

Justificación: el reporte agrupa por día y suma totales sobre muchos registros; lo hago en un procedimiento almacenado para optimizarlo y reutilizarlo.

1) El procedimiento almacenado creado en SQL Server

```

CREATE PROCEDURE usp_ReporteVentasPorFecha
    @fecini DATE, @fecfin DATE
AS
BEGIN
    SET NOCOUNT ON;
    SELECT CONVERT(date, v.Fecha) AS Dia,
           COUNT(*) AS NumVentas,
           SUM(v.Total) AS TotalDia
    FROM Venta v
    WHERE v.Fecha BETWEEN @fecini AND @fecfin
    GROUP BY CONVERT(date, v.Fecha)
    ORDER BY Dia;
END

```

2) Cómo lo invoco desde el proyecto (EF Core)

```

// en el DbContext: modelBuilder.Entity<ReporteVentaDia>().HasNoKey();

// invocacion del SP desde el proyecto con FromSql
return await _ctx.ReporteVentas
    .FromSql($"EXEC usp_ReporteVentasPorFecha {ini}, {fin}")
    .ToListAsync();

```

D. AJAX (obligatorio) — buscar producto al registrar la venta

Justificación: al armar la venta, el vendedor escribe el código del producto y ve su nombre, precio y stock al instante, sin recargar la página.

1) Handler en el code-behind (Razor Pages)

```

// handler AJAX: devuelve el producto en JSON, sin recargar la pagina
public async Task<IActionResult> OnGetBuscarProductoAsync(int id)
{
    var p = await _ctx.Productos.FirstOrDefaultAsync(x => x.IdProducto == id);
    if (p == null) return new JsonResult(new { encontrado = false });
    return new JsonResult(new { encontrado = true, p.Nombre, p.Precio, p.Stock });
}

```

2) El fetch en la vista

```
// pide el producto al servidor y actualiza solo esos campos, sin recargar
const r = await fetch(`?handler=BuscarProducto&id=${id}`);
const data = await r.json();
if (data.encontrado) {
  document.getElementById("nombre").textContent = data.nombre;
  document.getElementById("precio").textContent = "$/ " + data.precio;
  document.getElementById("stock").textContent = data.stock;
}
```

RECUERDA este ejemplo es solo una referencia. En tu entrega, reemplaza el código por el de TU proyecto y explica con tus palabras qué hace cada parte y por qué elegiste esa tecnología.